The Queue Data Structure : (Principle → {FIFO})

* Implementation Using Array : →

arr — front (-1), rear 0 — f/r

Two variables to check elements : → front & rear

* Empty Queue : → front = rear = -1;

```
isEmpty( ) {
    return (front == -1 && rear == -1);
}
isFull() {
    return rear == MAX-SIZE - 1;
}
```

Add or Enqueue : →
(i) Full → can't add
(ii) Empty → front = rear = 0
(iii) Multiple → rear ++

Remove or Dequeue : →
(i) Empty → Can't remove
(ii) Single → front = rear = -1
(iii) Multiple → front ++

```
public int getSize( ) {
    → if( isEmpty) return 0;
      return r - f + 1;
}
```

empty: f = -1, r = -1 — $r - f + 1 = -1 - (-1) + 1 = -1 + 1 + 1 = 1$

single: f = 0, r = 0 — $0 - 0 + 1 = 1$

f, r: x y z w — 0 1 2 3 — $3 - 0 + 1 = 4$

Input = N = 5 Expected Output : 1 10 11 100 101
Queue <String> = ["1"];

Step 1 : Pop "1" Print "1" Push "10", "11" {concatenation} ["10", "11"]
Step 2 : Pop "10" Print "10" Push "100", "101" ["11" "100" "101"]
Step 3 : Pop "11" Print "11" Push "110", "111" ["100" "101" "110" "111"]
Step 4 : Pop "100" Print "100" Push "1000", "1001" [101 110 111 1000 1001]
Step 5 : Pop "101" Print "101" Push "1010", "1011" [110 111 1000 1001 1010 1011]
(N = 5) stop

Reversal of Functionalities : →

1. Stack Using Queue

→ Stack is LIFO
→ Queue is FIFO

So, while pushing, we force the new element to come to the front.

Logic : → q1 & q2

* Add new element to q2
* Move all elements of q1 into q2
* Swap q1 and q2

Now: newest element is at front of q1, so pop() becomes easy.